

UNIT – 1

Introduction

Knowledge representation and reasoning (KRR, KR&R, KR²)

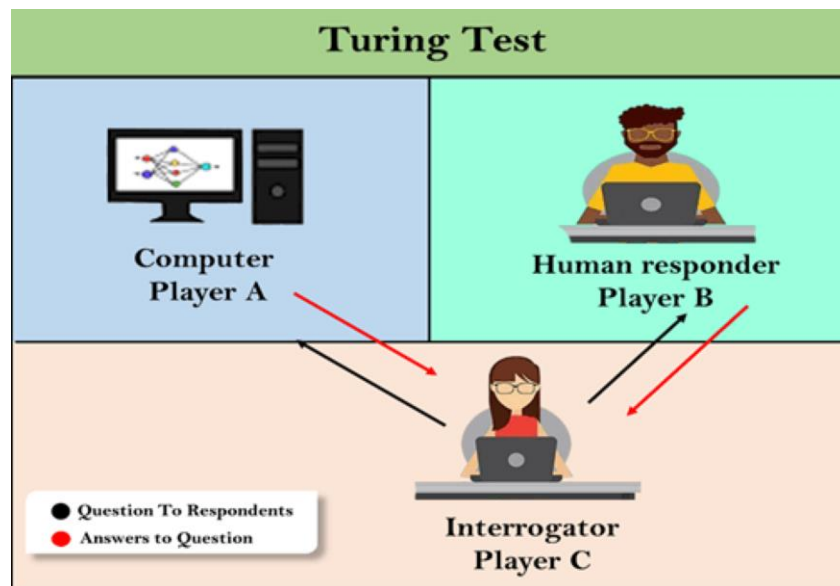
Knowledge representation and reasoning (KRR, KR&R, KR²) is a field of artificial intelligence that focuses on designing computer representations that capture information about the world that can be used for solving complex problems.

The earliest work in computerized knowledge representation was focused on general problem-solvers such as the General Problem Solver (GPS) system developed by Allen Newell and Herbert A. Simon in 1959. These systems featured data structures for planning and decomposition. The system would begin with a goal. It would then decompose that goal into sub-goals and then set out to construct strategies that could accomplish each subgoal.

Turing Test in AI

In **1950**, Alan Turing introduced a test to check whether a **machine can think like a human or not**, this test is known as the **Turing Test**. In this test, Turing proposed that the computer can be said to be an intelligent if it can mimic human response under specific conditions.

Turing Test was introduced by Turing in his 1950 paper, "Computing Machinery and Intelligence," which considered the question, "**Can Machine think?**"



The Turing test is based on a party game "Imitation game," with some modifications.

This game involves three players in which one player is Computer, another player is human responder, and the third player is a human Interrogator, who is isolated from other two players and his job is to find that which player is machine among two of them.

Consider, Player A is a computer, Player B is human, and Player C is an interrogator. Interrogator is aware that one of them is machine, but he needs to identify this on the basis of questions and their responses.

The conversation between all players is via keyboard and screen so the result would not depend on the machine's ability to convert words as speech.

The test result does not depend on each correct answer, but only how closely its responses like a human answer. The computer is permitted to do everything possible to force a wrong identification by the interrogator.

The questions and answers can be like:

Interrogator: Are you a computer?

PlayerA (Computer): No

Interrogator: Multiply two large numbers such as $(256896489 \times 456725896)$

Player A: Long pause and give the wrong answer.

In this game, if an interrogator would not be able to identify which is a machine and which is human, then the computer passes the test successfully, and the machine is said to be intelligent and can think like a human.

"In 1991, the New York businessman Hugh Loebner announces the prize competition, offering a \$100,000 prize for the first computer to pass the Turing test. However, no AI program to till date, come close to passing an undiluted Turing test".

Features required for a machine to pass the Turing test:

- **Natural language processing:** NLP is required to communicate with Interrogator in general human language like English.
- **Knowledge representation:** To store and retrieve information during the test.
- **Automated reasoning:** To use the previously stored information for answering the questions.
- **Machine learning:** To adapt new changes and can detect generalized patterns.

- **Vision (For total Turing test):** To recognize the interrogator actions and other objects during a test.
- **Motor Control (For total Turing test):** To act upon objects if requested.

Advantages of the Turing Test in Artificial Intelligence:

- **Evaluating machine intelligence:** The Turing Test provides a simple and well-known method for evaluating the intelligence of a machine.
- **Setting a benchmark:** The Turing Test sets a benchmark for artificial intelligence research and provides a goal for researchers to strive towards.
- **Inspiring research:** The Turing Test has inspired numerous studies and experiments aimed at developing machines that can pass the test, which has driven progress in the field of artificial intelligence.
- **Simple to administer:** The Turing Test is relatively simple to administer and can be carried out with just a computer and a human judge.

Disadvantages of the Turing Test in Artificial Intelligence:

- **Limited scope:** The Turing Test is limited in scope, focusing primarily on language-based conversations and not taking into account other important aspects of intelligence, such as perception, problem-solving, and decision-making.
- **Human bias:** The results of the Turing Test can be influenced by the biases and preferences of the human judge, making it difficult to obtain objective and reliable results.
- **Not representative of real-world AI:** The Turing Test may not be representative of the kind of intelligence that machines need to demonstrate in real-world applications.

Physical Symbol System

A physical symbol system (also called a formal system) takes physical patterns (symbols), combining them into structures (expressions) and manipulating them (using processes) to produce new expressions.

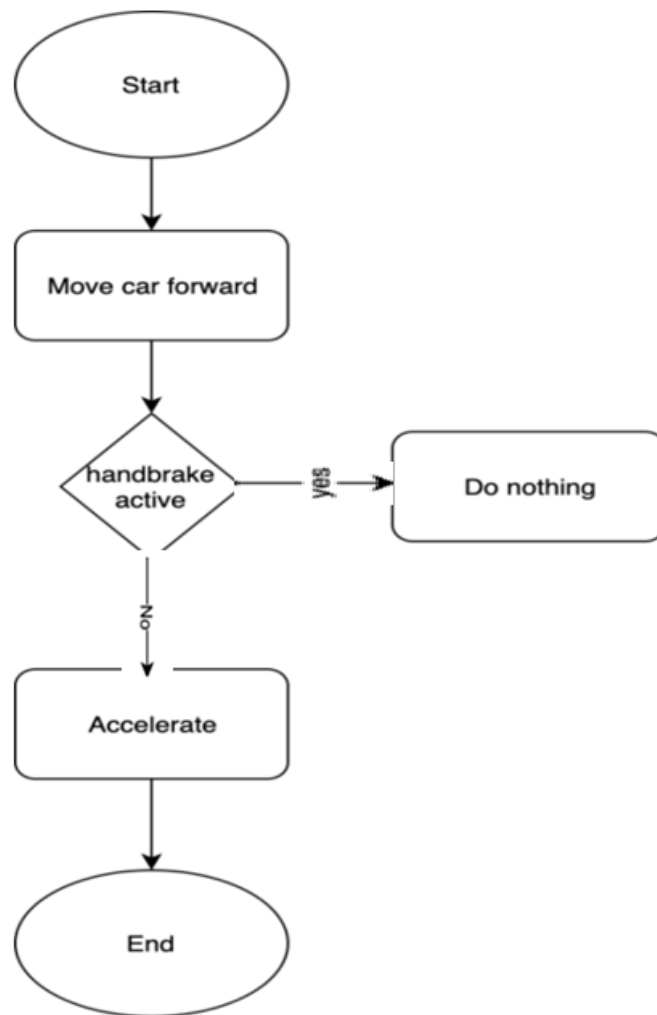
Scope of Symbolic AI

Symbolic AI is an approach that trains Artificial Intelligence (AI) the same way **human**

brain learns. Symbolic AI has been **applied** in various fields, including natural language processing, expert systems, and robotics.

Flowcharts can depict the logic of symbolic AI programs very clearly

Symbolic artificial intelligence showed early progress at the dawn of AI and computing. You can **easily visualize the logic of rule-based programs, communicate them, and troubleshoot them.**



Medical diagnosis systems use Symbolic AI to provide recommendations to doctors based on patient symptoms. Autonomous cars use Symbolic AI to make decisions based on the environment, such as recognizing stop signs and traffic lights.

What is an Agent?

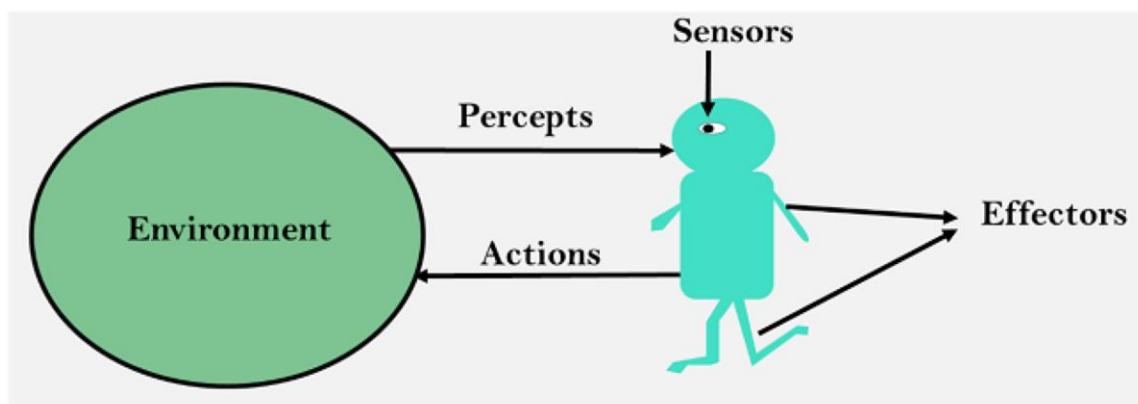
An **agent** can be anything that **perceive its environment through sensors** and act upon that environment through actuators. An Agent runs in the cycle of perceiving, thinking, and acting. An agent can be:

- **Human-Agent:** A human agent has eyes, ears, and other organs which work for sensors and hand, legs, vocal tract work for actuators.
- **Robotic Agent:** A robotic agent can have cameras, infrared range finder, NLP for sensors and various motors for actuators.
- **Software Agent:** Software agent can have keystrokes, file contents as sensory input and act on those inputs and display output on the screen.

Hence the world around us is full of agents such as **thermostat, cellphone, camera**, and even we are also agents.

Before moving forward, we should first know about sensors, effectors, and actuators.

- **Sensor:** Sensor is a device which detects the change in the environment and sends the information to other electronic devices. An agent observes its environment through sensors.
- **Actuators:** Actuators are the component of machines that converts energy into motion. The actuators are only responsible for moving and controlling a system. An actuator can be an electric motor, gears, rails, etc.
- **Effectors:** Effectors are the devices which affect the environment. Effectors can be legs, wheels, arms, fingers, wings, fins, and display screen.



State space search

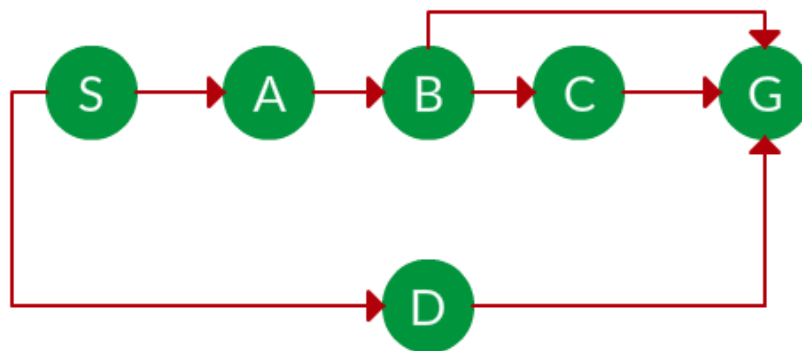
State space search in Knowledge Representation and Reasoning (KRR) is a method where computers explore different scenarios to find solutions. It starts from a given situation (initial state), applies possible actions (operators) to move to new situations (states), and continues until the desired outcome (goal state) is achieved. This technique helps computers solve problems by trying out different options step by step.

Depth-first search (DFS)

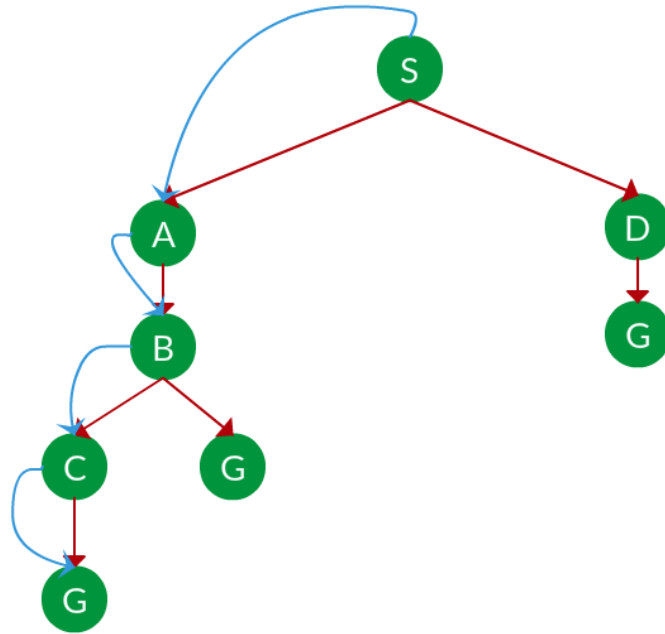
Depth-first search (DFS) is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking. It uses last in- first-out strategy and hence it is implemented using a stack.

Example:

Question. Which solution would DFS find to move from node S to node G if run on the graph below?



Solution. The equivalent search tree for the above graph is as follows. As DFS traverses the tree “deepest node first”, it would always pick the deeper branch until it reaches the solution (or it runs out of nodes, and goes to the next branch). The traversal is shown in blue arrows.



Path: S -> A -> B -> C -> G

d = the depth of the search tree = the number of levels of the search tree.

n^i = number of nodes in level i .

Time complexity: Equivalent to the number of nodes traversed in DFS.

$$T(n) = 1 + n^1 + n^2 + n^3 + \dots + n^d = O(n^d)$$

Space complexity: Equivalent to how large can the fringe get. $S(n) = O(n \times d)$

Completeness: DFS is complete if the search tree is finite, meaning for a given finite search tree, DFS will come up with a solution if it exists.

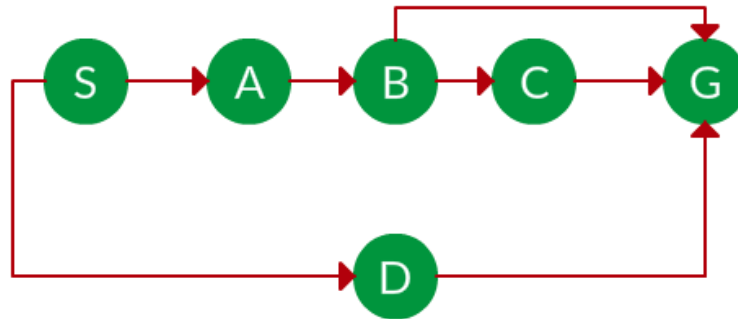
Optimality: DFS is not optimal, meaning the number of steps in reaching the solution, or the cost spent in reaching it is high.

Breadth First Search:

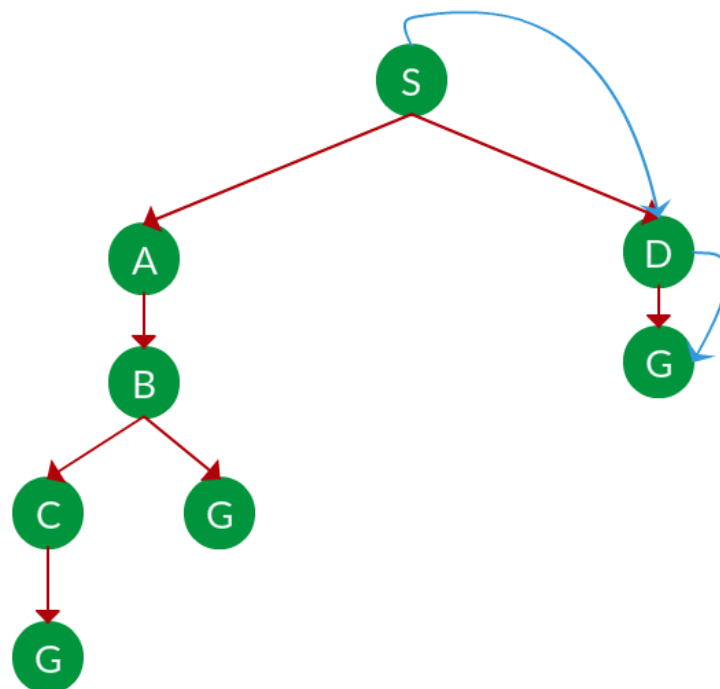
Breadth-first search (BFS) is an algorithm for traversing or searching tree or graph data structures. It starts at the tree root (or some arbitrary node of a graph, sometimes referred to as a 'search key'), and explores all of the neighbour nodes at the present depth prior to moving on to the nodes at the next depth level. It is implemented using a queue.

Example:

Question. Which solution would BFS find to move from node S to node G if run on the graph below?



Solution. The equivalent search tree for the above graph is as follows. As BFS traverses the tree “shallowest node first”, it would always pick the shallower branch until it reaches the solution (or it runs out of nodes, and goes to the next branch). The traversal is shown in blue arrows.



Path: S -> D -> G

s = the depth of the shallowest solution.

n^l = number of nodes in level l .

Time complexity: Equivalent to the number of nodes traversed in BFS until the shallowest solution.

$$T(n) = 1 + n^2 + n^3 + \dots + n^s = O(n^s)$$

Space complexity: Equivalent to how large can the fringe get. $\overline{S(n)} = O(n^s)$

Completeness: BFS is complete, meaning for a given search tree, BFS will come up with a solution if it exists.

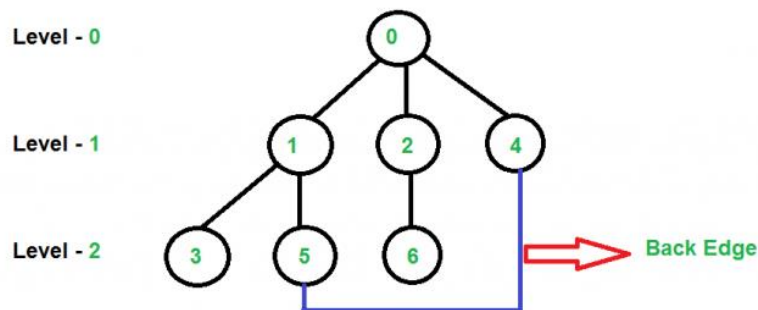
Optimality: BFS is optimal as long as the costs of all edges are equal.

Depth First Iterative Deepening Search

IDDFS combines depth-first search's space-efficiency and breadth-first search's fast search (for nodes closer to root).

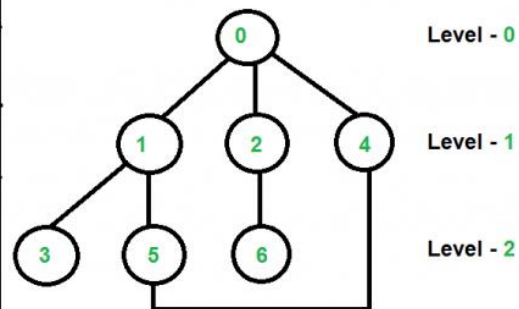
Illustration: There can be two cases:

1. **When the graph has no cycle:** This case is simple. We can DFS multiple times with different height limits.
2. **When the graph has cycles.** This is interesting as there is no visited flag in IDDFS.



Although, at first sight, it may seem that since there are only **3 levels**, so we might think that **Iterative Deepening Depth First Search** of level 3, 4, 5,...and so on will remain same. But, this is not the case. You can see that there is a **cycle** in the above graph, hence **IDDFS** will change for level-3, 4, 5..and so on.

Depth	Iterative Deepening Depth First Search
0	0
1	0 1 2 4
2	0 1 3 5 2 6 4 5
3	0 1 3 5 4 2 6 4 5 1



The explanation of the above pattern is left to the readers.

Time Complexity: Suppose we have a tree having branching factor 'b' (number of children of each node), and its depth 'd', i.e., there are b^d nodes. In an iterative deepening search, the nodes on the bottom level are expanded once, those on the next to bottom level are expanded twice, and so on, up to the root of the search tree, which is expanded $d+1$ times. So the total number of expansions in an iterative deepening search is-

$$(d)b + (d-1)b^2 + \dots + 3b^{d-2} + 2b^{d-1} + b^d$$

That is,

Summation $[(d + 1 - i) b^i]$, from $i = 0$ to $i = d$

Which is same as $O(b^d)$

After evaluating the above expression, we find that asymptotically IDDFS takes the same time as that of DFS and BFS, but it is indeed slower than both of them as it has a higher constant factor in its time complexity expression. IDDFS is best suited for a complete infinite tree

Complexity:

Time: $O(b^d)$

Space: $O(d)$

A comparison table between DFS, BFS and IDDFS

	Time Complexity	Space Complexity	When to Use ?
DFS	$O(b^d)$	$O(d)$	=> Don't care if the answer is closest to the starting vertex/root. => When graph/tree is not very big/infinite.
BFS	$O(b^d)$	$O(b^d)$	=> When space is not an issue => When we do care/want the closest answer to the root.
IDDFS	$O(b^d)$	$O(bd)$	=> You want a BFS, you don't have enough memory, and somewhat slower performance is accepted. In short, you want a BFS + DFS.